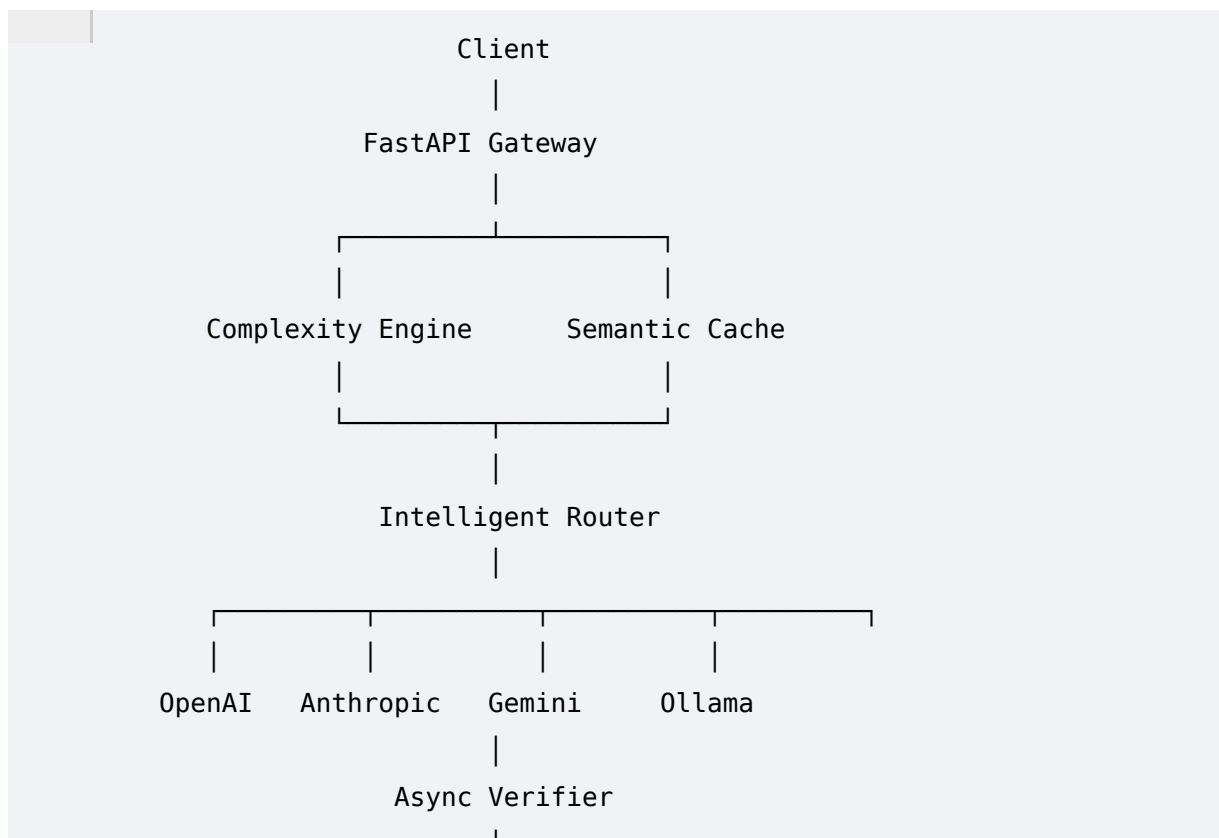


I actually like this project a lot, but **I would build a V2 of it**, not exactly as written.

The roadmap you've posted is solid (I'd rate it **8.5/10**), but there are a few things that make it look like an academic project rather than something an AI infrastructure engineer would build in 2026.

Below is how I'd improve it to make it portfolio-worthy and something that could genuinely impress recruiters.

Overall Architecture



This looks much more like a real AI gateway.

Phase 0 (Before Writing Any Code)

Most people skip this.

Create a design document first.

Include:

- Problem Statement
- Architecture
- Routing Logic

- Cost Formula
- Quality Metric
- Failure Cases
- Future Improvements

This alone makes your GitHub look significantly more professional.

Phase 1 — Provider Layer (2–3 Days)

Instead of hardcoding providers, make them pluggable.

```
providers/  
  
base.py  
  
openai_provider.py  
  
anthropic_provider.py  
  
gemini_provider.py  
  
ollama_provider.py
```

Every provider implements:

```
class BaseProvider:  
  
    async def chat():  
  
        pass  
  
    async def embeddings():  
  
        pass  
  
    async def health():
```

This means adding a new provider later requires almost no changes.

Phase 2 — Model Registry

Instead of a Python dictionary...

Use YAML.

```
gpt-4o-mini:

  provider: openai

  input_cost: 0.15

  output_cost: 0.60

  latency: medium

  quality: 8.5

  context: 128000
```

Why?

Because production systems rarely require code changes to update pricing.

Phase 3 — Complexity Engine

This is where the project becomes interesting.

Don't use only Logistic Regression.

Build **three versions**.

Version 1

Rule Engine

Example:

```
Prompt Length

Contains Code

Contains JSON

Contains Math

Contains "Explain"

Contains "Analyze"
```

Number of Constraints

↓

Complexity Score

Version 2

ML

Random Forest

or

LightGBM

Version 3

Sentence Embeddings

Prompt

↓

Embedding

↓

Small classifier

Recruiters love seeing progression from heuristic → ML.

Phase 4 — Intelligent Router

Instead of

Tier 1

↓

Cheap Model

Make routing based on **multiple factors**.

Example:

Complexity
Prompt Length
Estimated Tokens
Current Provider Latency
Provider Health
Current Rate Limit
Model Cost
Cache Hit Probability

↓

Best Model

This feels much closer to how production AI platforms route traffic.

Phase 5 — Semantic Cache ★

This is the feature that can save the most money.

Instead of checking for exact prompts...

Compute embeddings.

User:
Explain recursion
↓
Embedding
↓
Vector Search
↓

95% Similar

↓

No LLM call.

Use:

- Redis
- pgvector

or

- FAISS

This demonstrates practical understanding of AI infrastructure.

Phase 6 — Shadow Evaluation

This is the strongest part of the project.

Don't evaluate every request.

Evaluate only a sample (e.g., 5%).

Cheap Model

↓

User gets response

↓

Background Job

↓

Premium Model

↓

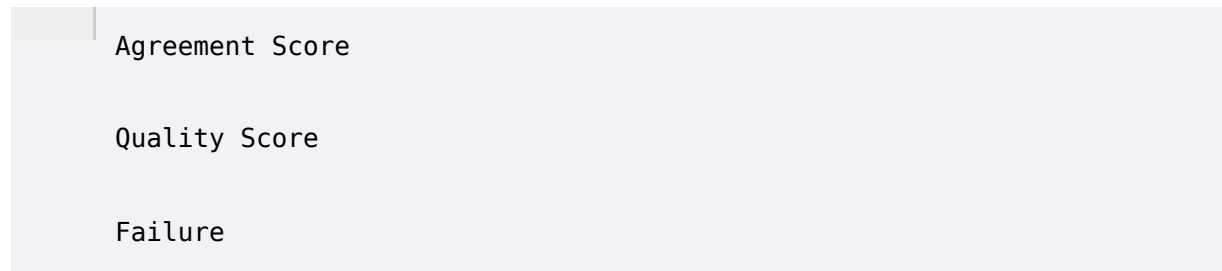
Compare

Metrics:

- BERTScore
- Cosine Similarity

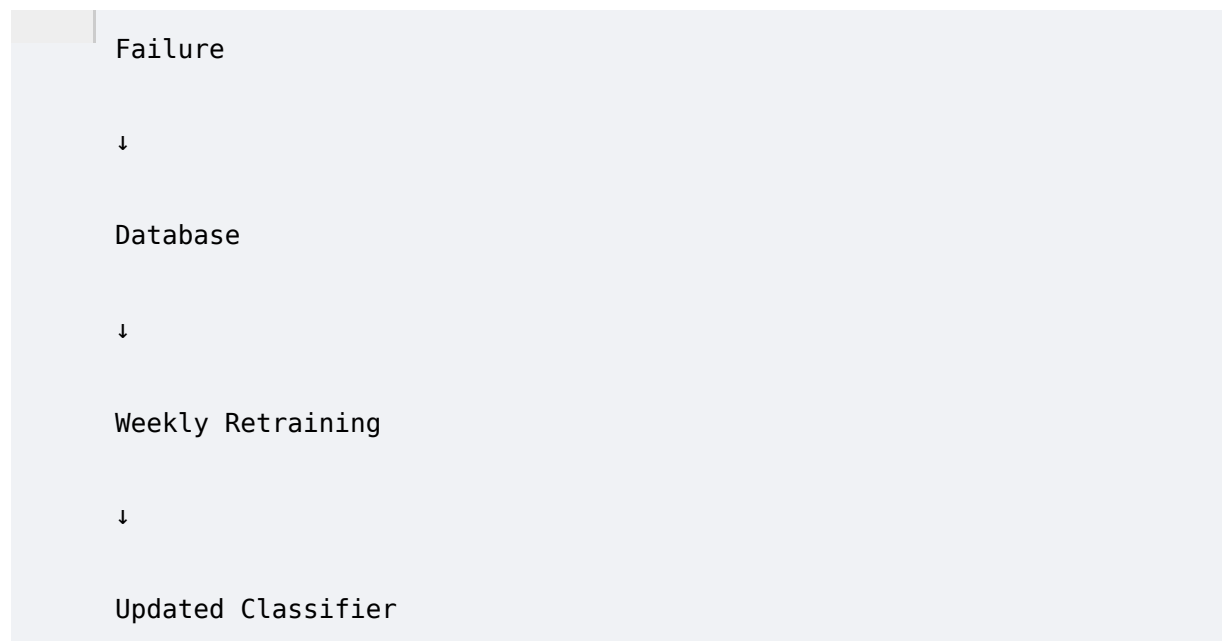
- LLM-as-a-Judge

Store:



Phase 7 — Feedback Loop

Every routing failure becomes training data.



This creates a system that improves over time.

Phase 8 — Observability

Most student projects miss this.

Log:

- Latency
- Provider Errors
- Cost
- Cache Hits
- Tokens
- Retries

- Escalations

Use:

- OpenTelemetry
- Prometheus
- Grafana

This immediately gives the project a production feel.

Phase 9 — Dashboard

Instead of just charts...

Add business KPIs.

Display:

- Today's Cost
- Total Savings
- Average Latency
- Cache Hit Rate
- Escalation Rate
- Requests per Minute
- Provider Availability
- Most Expensive Prompts
- Top Cached Prompts

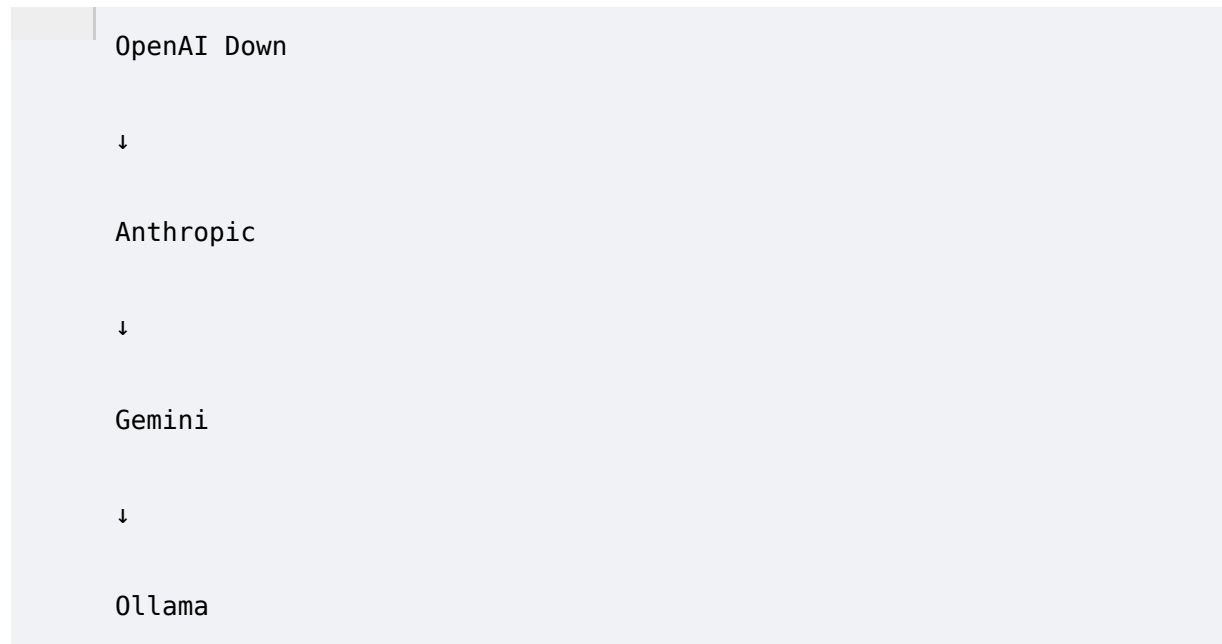
This tells a story beyond the technology.

Phase 10 — Production Features

Include:

- JWT Authentication
- API Keys
- Rate Limiting
- Retry Logic
- Circuit Breaker
- Streaming Responses
- Request Timeout
- Fallback Provider

Example:



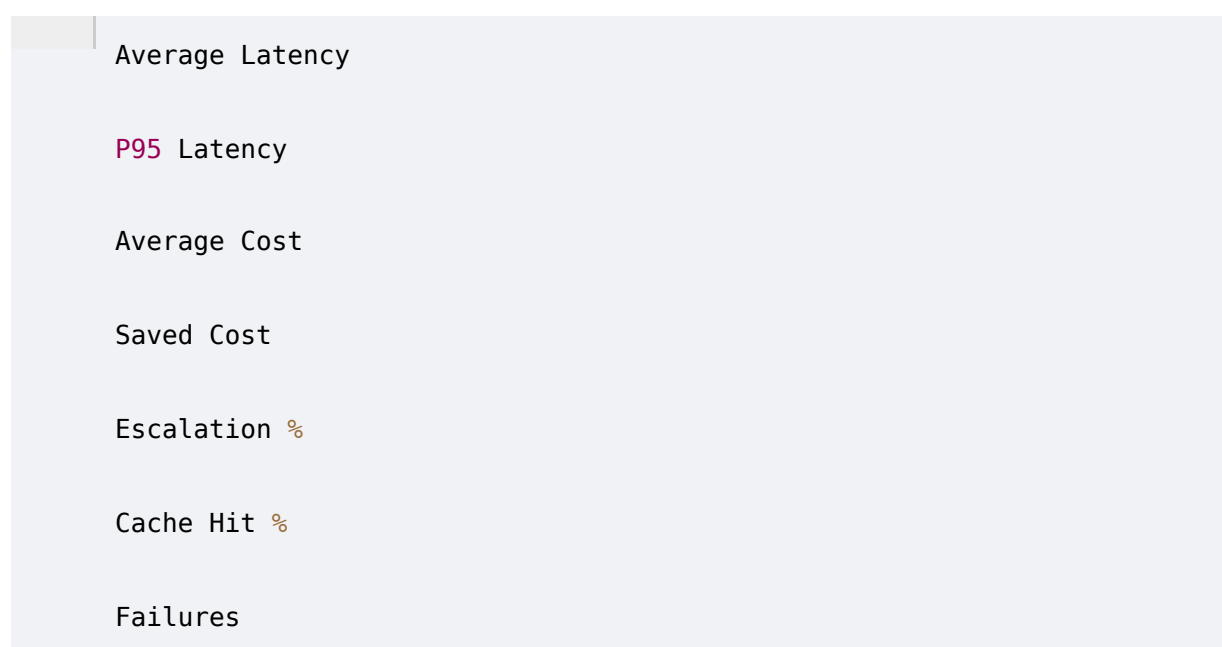
This is a real production pattern.

Phase 11 — Load Testing

Run:

- 1,000 requests
- 5,000 requests
- Mixed workloads

Collect:



These numbers make excellent portfolio material.

Suggested Tech Stack

Layer	Technology
Backend	FastAPI
Async Tasks	Celery + Redis (or Dramatiq)
Database	PostgreSQL
Vector Search	pgvector
Cache	Redis
ML	scikit-learn + LightGBM
Embeddings	sentence-transformers
Monitoring	Prometheus + Grafana + OpenTelemetry
Dashboard	React + TypeScript (or

	Streamlit for a quick prototype)
Deployment	Docker Compose
CI/CD	GitHub Actions

Suggested Folder Structure

```

llm-cost-autopilot/
|
├─ api/
├─ router/
├─ providers/
├─ complexity/
|   ├─ rules.py
|   ├─ classifier.py
|   └─ features.py
├─ cache/
├─ validator/
├─ analytics/
├─ pricing/
├─ database/
├─ worker/
├─ dashboard/
├─ monitoring/
└─ tests/

```

What Recruiters Want to Read

Instead of saying:

■ "Built an LLM router."

Say:

Designed and implemented an intelligent multi-provider LLM gateway that dynamically routes requests based on prompt complexity, provider health, latency, and pricing, reducing inference cost by 50–70% while maintaining over 95% quality parity through asynchronous shadow evaluation and automated feedback-driven routing improvements.

That single sentence communicates:

- AI engineering
- Backend engineering
- Distributed systems
- MLOps
- LLMOps
- Cost optimization
- Observability
- Production thinking

This is the kind of project that can genuinely stand out for AI platform, ML infrastructure, and backend engineering internships if it's implemented well rather than just described.